

Week-4

June 11, 2026

1 Import the packages

```
[1]: import pandas as pd
import numpy as np
from scipy.integrate import quad
from scipy.optimize import minimize
import scipy.linalg as la # For safe matrix inversion
```

2 Load the Dataset

```
[2]: data_file_path = 'data/Pantheon+SH0ES.dat'
try:
    df = pd.read_csv(data_file_path, sep=r'\s+')
    print(f"Dataset loaded successfully. Total supernovae: {len(df)}")
except FileNotFoundError:
    print(f"Error: The file {data_file_path} was not found.")
```

Dataset loaded successfully. Total supernovae: 1701

3 Task 1, 2, 3 & 4 - Load the Covariance Matrix

```
[3]: cov_file_path = 'data/Pantheon+SH0ES_STAT+SYS.cov'

try:
    # Read the covariance matrix file.
    # In cosmology, these are often flattened 1D arrays or matrices.
    with open(cov_file_path, 'r') as f:
        lines = f.readlines()

    # Sometimes the first line is just the dimension 'N'. If so, skip it.
    if len(lines[0].split()) == 1:
        lines = lines[1:]

    # Load into a 1D numpy array and then reshape to NxN
    cov_1d = np.loadtxt(lines)
    N = len(df)
```

```

# Task 2: Verify dimensions
if len(cov_1d) == N * N:
    C = cov_1d.reshape((N, N))
    print(f"Covariance matrix loaded and reshaped to {C.shape}.")
elif cov_1d.shape == (N, N):
    C = cov_1d
    print(f"Covariance matrix loaded successfully with shape {C.shape}.")
else:
    print(f"Error: Dimension mismatch. Expected {N}x{N} elements, got {cov_1d.shape}.")

except FileNotFoundError:
    print(f"Error: The file {cov_file_path} was not found.")

```

Covariance matrix loaded and reshaped to (1701, 1701).

4 Task 5 & 6 - Invert Covariance and Compute Log-Determinant

```

[4]: print("Performing Cholesky decomposition...")

# Cholesky decomposition: C = L @ L.T
# This is much safer and faster for large covariance matrices
c_chol, lower = la.cho_factor(C)

# Invert the covariance matrix (C^-1)
C_inv = la.cho_solve((c_chol, lower), np.eye(N))
print("Covariance matrix successfully inverted.")

# Safely compute the log-determinant
# det(C) = Product of squared diagonal elements of the Cholesky factor L
# ln(det(C)) = 2 * sum(ln(diag(L)))
log_det_C = 2.0 * np.sum(np.log(np.diag(c_chol)))
print(f"Log-determinant of C calculated: {log_det_C:.2f}")

```

Performing Cholesky decomposition...
Covariance matrix successfully inverted.
Log-determinant of C calculated: -6400.75

5 Theoretical Physics Models

```

[5]: # Speed of light in km/s
c = 299792.458

def model_calibrators(M_B, mu_ceph):
    return M_B + mu_ceph

```

```

def E_inverse(z, Omega_m):
    Omega_L = 1.0 - Omega_m
    return 1.0 / np.sqrt(Omega_m * (1 + z)**3 + Omega_L)

def luminosity_distance_mpc(z_hel, z_cmb, H_0, Omega_m):
    integral, _ = quad(E_inverse, 0, z_cmb, args=(Omega_m,))
    return (c / H_0) * (1 + z_hel) * integral

def model_cosmology(z_hel, z_cmb, M_B, H_0, Omega_m):
    d_L_mpc = luminosity_distance_mpc(z_hel, z_cmb, H_0, Omega_m)
    mu_lcdm = 5.0 * np.log10(d_L_mpc) + 25.0
    return M_B + mu_lcdm

```

6 Task 7 - Build the Full Gaussian Log-Likelihood

```

[6]: def build_residual_vector(params, data):
    """Computes the vector Delta = m_b,obs - m_b,th"""
    Omega_m, H_0, M_B = params
    N_data = len(data)
    delta = np.zeros(N_data)

    for i, row in data.iterrows():
        m_b_obs = row['m_b_corr']
        is_calib = row['IS_CALIBRATOR']

        if is_calib == 1:
            mu_ceph = row['CEPH_DIST']
            if mu_ceph > -9: # Checking against placeholder -9
                m_b_th = model_calibrators(M_B, mu_ceph)
            else:
                m_b_th = m_b_obs # Fallback if missing
        else:
            m_b_th = model_cosmology(row['zHEL'], row['zCMB'], M_B, H_0,
↪Omega_m)

        delta[i] = m_b_obs - m_b_th

    return delta

def negative_log_likelihood(params, data, C_inv, log_det_C):
    """
    Computes -ln(L).
    We return the negative because scipy 'minimizes' functions. Minimizing
↪-ln(L) maximizes ln(L).
    """

```

```

Omega_m, H_0, M_B = params

# Hard physical boundaries to prevent integration errors
if Omega_m <= 0 or Omega_m >= 1 or H_0 <= 0:
    return np.inf

# 1. Build the residual vector Delta
delta = build_residual_vector(params, data)

# 2. Compute chi-square:  $\Delta^T * C^{-1} * \Delta$ 
chi2 = np.dot(delta.T, np.dot(C_inv, delta))

# 3. Full Log-Likelihood
N_obs = len(data)
ln_L = -0.5 * (chi2 + log_det_C + N_obs * np.log(2 * np.pi))

return -ln_L # Return negative for the minimizer

```

7 Task 8 & 9 - Test and Optimize for Best Fit

```

[7]: print("Starting maximum likelihood estimation (minimizing negative_
      ↪ log-likelihood)...")

# Initial guess: [Omega_m, H_0, M_B]
initial_guess = [0.5, 68.0, 0]
bounds = ((0.01, 0.99), (50.0, 100.0), (-21.0, -18.0))

# Run the optimization
result = minimize(
    negative_log_likelihood,
    x0=initial_guess,
    args=(df, C_inv, log_det_C),
    method='L-BFGS-B',
    bounds=bounds
)

if result.success:
    best_ln_L = -result.fun # Convert back to positive log-likelihood
    print("\nOptimization Successful!")
    print(f"Maximum Log-Likelihood: {best_ln_L:.2f}")
    print("\nBest-Fit Parameters:")
    print(f" - Matter Density (Omega_m) = {result.x[0]:.4f}")
    print(f" - Hubble Constant (H_0) = {result.x[1]:.2f} km/s/Mpc")
    print(f" - Absolute Magnitude (M_B) = {result.x[2]:.4f}")
else:
    print("Optimization Failed.")

```

```
print(result.message)
```

Starting maximum likelihood estimation (minimizing negative log-likelihood)...

Optimization Successful!

Maximum Log-Likelihood: 846.94

Best-Fit Parameters:

- Matter Density (Ω_m) = 0.3479
- Hubble Constant (H_0) = 72.87 km/s/Mpc
- Absolute Magnitude (M_B) = -19.2505

```
[ ]:
```